

Documentação e Guia Prático: Cilium Tetragon

1 Overview

O componente Cilium Tetragon permite uma segurança baseada em eBPF com observabilidade em tempo real e aplicação em tempo de execução.

O Tetragon detecta e é capaz de reagir a eventos de segurança, como:

- Eventos de execução de processos;
- Atividade de chamada de sistema (*syscalls*);
- Atividade de E/S, incluindo acesso à rede e a arquivos.

Quando utilizado em um ambiente Kubernetes, ele compreende identidades nativas (como *namespaces*, *pods*, etc.) de modo que a detecção de eventos de segurança possa ser configurada individualmente para cada carga de trabalho.

2 Visão Geral das Funcionalidades

2.1 eBPF em Tempo Real

O Tetragon é uma ferramenta de aplicação de segurança e observabilidade em tempo de execução. Isso significa que o Tetragon aplica políticas e filtragem diretamente via eBPF no *kernel*. Ele realiza a filtragem, o bloqueio e a reação a eventos diretamente no *kernel*, em vez de enviar todos os eventos para um agente no espaço do usuário.

Para um caso de uso de observabilidade, a aplicação de filtros diretamente no *kernel* reduz drasticamente a sobrecarga de observação. Ao evitar trocas de contexto e ativações dispendiosas — especialmente para eventos de alta frequência como operações de envio, leitura ou gravação —, o eBPF poupa recursos essenciais.

O Tetragon oferece filtros avançados (arquivo, soquete, nomes binários, *namespaces*/capacidades, etc.) no eBPF, permitindo que os usuários especifiquem apenas os eventos importantes em seu contexto e encaminhem exclusivamente esses registros ao agente no espaço do usuário.

2.2 Flexibilidade do eBPF

O Tetragon pode se integrar a qualquer função do *kernel* Linux e filtrar eventos com base em seus argumentos, valor de retorno e metadados associados (por exemplo, nomes de executáveis, arquivos e outras propriedades).

Ao criar políticas de rastreamento (*tracing policies*), os usuários podem resolver diversos cenários de segurança e observabilidade. Nenhum detalhe específico sobre quais funções

são monitoradas ou quais filtros são aplicados está fixado rigidamente no mecanismo: os usuários têm liberdade para rastrear funções do *kernel* personalizadas.

Além disso, a integração profunda no *kernel* garante que as estruturas de dados não possam ser adulteradas por binários no espaço do usuário, evitando problemas comuns como leitura incorreta de chamadas de sistema (*syscalls*), modificações maliciosas antes da validação (*race conditions*/TOCTOU) ou falhas de página.

2.3 Compatível com o Kernel eBPF

Por meio do eBPF, o Tetragon possui visibilidade total sobre o estado interno do *kernel* Linux. O sistema combina essas métricas com o contexto do Kubernetes ou regras do usuário para aplicar políticas em tempo real.

Isso possibilita restringir capacidades de processos, descritores de arquivos e soquetes de rede dinamicamente. Caso uma aplicação altere privilégios indevidamente, é possível acionar alertas em tempo real ou encerrar (*kill*) o processo imediatamente antes da conclusão da chamada de sistema maliciosa.

3 Especificação de Ambiente (Componentes do Projeto)

Abaixo estão detalhados os recursos da infraestrutura de hardware, sistema operacional e ferramentas de runtime configuradas para hospedar o Tetragon de forma adequada.

3.1 Proxmox VE e Máquina Virtual

- **Hipervisor:** Proxmox VE 9.2.0 (pve-manager: 9.2.3)
- **Kernel do Host:** Linux 7.0.12-1-pve
- **Hardware Físico:** Intel Core i3-6006U (2.00 GHz, 1 Socket, 4 Cores)
- **Tipo de Instância (Guest):** VM QEMU/KVM (ID 101)
- **Recursos da VM:** 2 vCPUs | 2.0 GiB RAM | 32.0 GiB Disco (Boot)

3.2 Sistema Operacional e Containers

- **Sistema Operacional (VM):** Ubuntu 26.04 LTS (Resolute Raccoon)
- **Kernel do Guest:** Linux 7.0.0-27-generic
- **Docker Engine:** 29.7.2 (API v1.55) | **Containerd:** v2.3.4 | **Runc:** 1.4.3
- **Recomendação Tetragon:** 256 MiB a 512 MiB de limite de memória.

4 Pré-requisitos do Ambiente

Antes da instalação, confirme que a máquina atende aos requisitos do Tetragon e que as ferramentas utilizadas ao longo deste guia estão presentes. Todos os comandos desta seção são de verificação e não alteram o sistema.

4.1 Requisitos do Kernel

O Tetragon depende de eBPF com informações de tipo (BTF) e de *cgroups* na versão 2.

```
uname -r
ls -l /sys/kernel/btf/vmlinux
stat -fc %T /sys/fs/cgroup
```

- `uname -r`: a versão do *kernel* deve ser 5.4 ou superior;
- `/sys/kernel/btf/vmlinux`: o arquivo precisa existir. É por meio dele que os programas BPF são adaptados ao *kernel* em uso;
- `stat -fc %T`: o resultado esperado é `cgroup2fs`, indicando *cgroups* unificados.

Nota: caso o arquivo BTF não exista, o kernel foi compilado sem `CONFIG_DEBUG_INFO_BTF` e o Tetragon não será capaz de carregar os sensores. Em distribuições recentes, como o Ubuntu 24.04 e o 26.04, a opção já vem habilitada.

4.2 Ferramentas Utilizadas

As ferramentas a seguir são usadas nas seções deste guia. A tabela indica onde cada uma aparece.

- `curl` e `wget`: download de políticas, da CLI e dos arquivos do aplicativo de demonstração;
- `jq`: formatação das saídas em JSON, na verificação de assinaturas e nos eventos;
- `cosign`: verificação da assinatura da imagem e do SBOM;
- `envsubst`: substituição de variáveis na política de rede. Faz parte do pacote `gettext-base`;
- `gpg` e `ca-certificates`: exigidos pela instalação do Docker Engine a partir do repositório oficial.

Verifique o que já está presente:

```
for cmd in curl wget jq cosign envsubst; do
    command -v $cmd >/dev/null && echo "OK: $cmd" || echo "FALTA: $cmd"
done
```

E instale o que faltar:

```
sudo apt update
sudo apt install -y curl wget jq cosign gettext-base ca-certificates
```

Nota: em instalações mínimas do Ubuntu, utilitários como `ping` (pacote `iputils-ping`) e `newgrp` (pacote `login`) podem não estar presentes. Nenhum dos dois é necessário para este guia.

4.3 Rede e Acesso

A instalação exige acesso à Internet para os domínios `download.docker.com`, `quay.io` e `github.com`. Verifique a resolução de nomes e a conectividade:

```
curl -sI https://download.docker.com | head -1
curl -sI https://quay.io | head -1
```

Nota: se a resposta for `Could not resolve host`, o problema é de DNS. Em servidores Ubuntu com `systemd-resolved`, a configuração é feita no `Netplan`, em `/etc/netplan/`, definindo `nameservers` e, no caso de endereçamento estático, também a rota padrão (routes com `to: default`). Aplique com `sudo netplan apply` e confirme com `resolvectl status` e `ip route`.

4.4 Portas Utilizadas

- 54321/tcp: API gRPC do agente, acessível apenas dentro do contêiner por padrão;
- 6789/tcp: servidor de saúde do agente;
- 2112/tcp: servidor de métricas, quando habilitado;
- 9090/tcp: interface do Prometheus, na seção de métricas;
- 8080/tcp: aplicativo de demonstração do quickstart.

5 Instalação do Docker Engine

Nesta documentação, o Tetragon é executado como um contêiner, então o Docker Engine precisa estar instalado na VM antes das próximas etapas. A instalação usa o repositório oficial da Docker, que oferece versões mais recentes do que os pacotes distribuídos pelo próprio Ubuntu ou Debian. Os comandos seguem a documentação oficial da Docker para Ubuntu (22.04, 24.04 e 26.04 LTS) e Debian (12 e 13).

Nota: os comandos abaixo começam com `sudo`, que executa a instrução com privilégios de administrador (root). Na primeira vez, o terminal pedirá a senha do seu usuário; enquanto você digita, nada aparece na tela, e isso é normal.

5.1 Remover Pacotes Conflitantes

Algumas distribuições oferecem pacotes não oficiais do Docker (como o `docker.io`) que entram em conflito com a versão oficial. Por isso, remova-os antes de instalar.

Ubuntu:

```
sudo apt remove $(dpkg --get-selections docker.io docker-compose \
  docker-compose-v2 docker-doc docker-buildx podman-docker \
  containerd runc | cut -f1)
```

Debian:

```
sudo apt remove $(dpkg --get-selections docker.io docker-compose \
  docker-doc docker-buildx podman-docker containerd runc | cut -f1)
```

Nota: se o sistema informar que nenhum desses pacotes está instalado, basta seguir para a próxima etapa. Imagens e contêineres antigos guardados em `/var/lib/docker/` não são apagados por este comando.

5.2 Instalar Dependências e Adicionar a Chave GPG

O `apt` é o gerenciador de pacotes do Ubuntu e do Debian: é ele que baixa e instala programas. Antes de adicionar o repositório da Docker, é preciso baixar a chave GPG da empresa, uma assinatura digital que permite ao `apt` confirmar que os pacotes vieram realmente da Docker e não foram alterados no caminho.

1. Atualize a lista de pacotes e instale as ferramentas necessárias (comum às duas distribuições):

```
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
```

- `apt update`: atualiza a lista de pacotes disponíveis;
- `apt install ca-certificates curl`: instala os certificados para conexões HTTPS e o `curl`, usado para baixar arquivos. Se o terminal perguntar se deseja continuar, confirme com `S` (ou `Y`, se o sistema estiver em inglês);
- `install -m 0755 -d`: cria a pasta `/etc/apt/keyrings`, onde a chave será guardada.

2. Baixe a chave GPG da Docker e libere a leitura do arquivo:

Ubuntu:

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
-o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Debian:

```
sudo curl -fsSL https://download.docker.com/linux/debian/gpg \
-o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

5.3 Adicionar o Repositório Oficial da Docker

Um repositório é um servidor de onde o `apt` baixa pacotes. O comando abaixo cria o arquivo `/etc/apt/sources.list.d/docker.sources`, que informa ao `apt` o endereço do repositório da Docker, a versão do sistema e a chave usada para validar os pacotes.

Nota: copie o bloco inteiro, da linha `sudo tee` até a linha `EOF`, e cole de uma só vez no terminal. É normal o conteúdo do arquivo ser exibido na tela logo em seguida.

Ubuntu:

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

Debian:

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

Em seguida, atualize a lista de pacotes para que o **apt** reconheça o novo repositório:

```
sudo apt update
```

*Nota: na saída desse comando devem aparecer linhas contendo **download.docker.com**. Se surgir um erro com esse endereço, revise a etapa anterior.*

5.4 Instalar o Docker Engine

Com o repositório configurado, instale os pacotes oficiais (o comando é o mesmo para Ubuntu e Debian):

```
sudo apt install docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

- **docker-ce**: o Docker Engine, serviço que executa os contêineres;
- **docker-ce-cli**: o comando **docker**, usado no terminal;
- **containerd.io**: o *runtime* de baixo nível (inclui **containerd** e **runc**);
- **docker-buildx-plugin**: ferramenta para construir imagens;
- **docker-compose-plugin**: habilita o comando **docker compose**, usado mais adiante para subir o aplicativo de demonstração.

5.5 Verificar a Instalação

1. Confira a versão instalada:

```
docker --version
```

A saída deve ser semelhante a (os números variam conforme a versão):

```
Docker version 29.7.2, build xxxxxxxx
```

2. Confirme que o serviço do Docker está em execução:

```
sudo systemctl status docker
```

Procure a indicação `active (running)` e pressione `q` para sair. Caso o serviço não esteja ativo, inicie-o manualmente:

```
sudo systemctl start docker
```

3. Execute o contêiner de teste:

```
sudo docker run hello-world
```

Esse comando baixa uma imagem de teste e a executa em um contêiner. Se tudo estiver correto, a saída conterá:

```
Hello from Docker!
```

```
This message shows that your installation appears to be working correctly.
```

Nota: por enquanto, os comandos `docker` precisam de `sudo`. A próxima seção mostra como adicionar seu usuário ao grupo `docker` para dispensar essa exigência.

6 Instalação do Tetragon como Contêiner

Como o ambiente deste projeto não utiliza Kubernetes, o Tetragon é implantado como contêiner ou como pacote do sistema. Aqui, segue-se com a implantação em contêiner.

Para executar uma versão estável, consulte as `tags` disponíveis no [repositório Tetragon no Quay](#) e selecione a desejada. Nesta documentação é utilizada a versão `v1.7.0`.

```
docker rm -f tetragon
docker run --name tetragon --rm -d \
  --pid=host --cgroupns=host --privileged \
  -v /sys/kernel/btf/vmlinux:/var/lib/tetragon/btf \
  quay.io/cilium/tetragon:v1.7.0
```

Nota: como o contêiner é iniciado com `--rm`, ele é removido ao ser parado. Para trocar a configuração ou as políticas, execute `docker rm -f tetragon` e repita o `docker run` com os novos parâmetros. Sem essa remoção, o Docker recusa a execução com `Conflict. The container name "/tetragon" is already in use.`

Nota: a tag da imagem é fixa (`v1.7.0`), portanto não se utiliza o sinalizador `--pull always`: ele consulta o registro a cada execução e faz o comando falhar quando não há acesso à Internet, mesmo com a imagem já presente localmente.

Nota: por padrão, o campo `node_name` dos eventos recebe o `hostname` do contêiner, que é o seu identificador aleatório. Para que os eventos sejam identificados pelo nome da máquina monitorada, acrescente `--hostname` ao `docker run` ou defina a variável `TETRAGON_NODE_NAME`. Em ambientes com várias máquinas, isso é o que permite correlacionar os eventos por origem.

7 Instalando a CLI Tetra

A CLI `tetra` está disponível para Linux (amd64 e arm64), macOS (amd64 e arm64) e Windows (amd64 e arm64). O comando abaixo instala a versão mais recente para Linux amd64:

```
curl -fsSL https://github.com/cilium/tetragon/releases/download/v1.7.0/tetra-linux-  
amd64.tar.gz | sudo tar -xz -C /usr/local/bin tetra
```

Nota: para instalar uma versão específica, consulte a página de [releases do Tetragon no GitHub](#).

7.1 Sinalizadores Globais da CLI

Na versão v1.7.1, utilizada neste ambiente, a CLI aceita os seguintes sinalizadores globais, válidos para qualquer subcomando:

- `--server-address`: endereço do servidor gRPC do agente;
- `--timeout`: tempo limite de conexão (padrão de 30 segundos);
- `--retries`: número de novas tentativas, com espera exponencial;
- `--max-recv-size`: tamanho máximo, em bytes, das mensagens gRPC recebidas;
- `-d, --debug`: habilita mensagens de depuração da própria CLI.

Os subcomandos disponíveis são `bugtool`, `cri`, `eventlog`, `explain`, `getevents`, `info`, `loglevel`, `policytest`, `probe`, `status`, `tracingpolicy` e `version`.

Nota: a documentação oficial descreve sinalizadores de cliente TLS (`--tls-ca-cert-files`, `--tls-cert-file`, `--tls-key-file`, `--tls-server-name` e `--tls-skip-verify`), porém eles ainda não existem no binário v1.7.1: o comando falha com `unknown flag`. Confirme sempre com `tetra --help` antes de utilizá-los. A conexão protegida por TLS é tratada na seção [Proteção da API gRPC com TLS](#).

Nota: o aviso `Could not find BPF map root`, exibido pela CLI, é esperado quando não há agente Tetragon em execução diretamente no host, e não impede o funcionamento dos comandos que usam `--server-address`.

8 Verificação da Instalação

A partir da versão 0.8.4, todas as imagens de contêiner do Tetragon são assinadas com [cosign](#). No Ubuntu 26.04, tanto o `cosign` quanto o `jq`, usado para formatar a saída, estão disponíveis nos repositórios oficiais:

```
sudo apt install cosign jq
```

8.1 Verificar a Assinatura da Imagem

```
cosign verify \  
--certificate-identity-regexp "https://github.com/cilium/tetragon/.*" \  
--certificate-github-workflow-repository cilium/tetragon \  
--certificate-oidc-issuer https://token.actions.githubusercontent.com \  
quay.io/cilium/tetragon:v1.7.0 | jq
```

Nota: a partir do cosign 2.x, a verificação sem chave exige `--certificate-identity` ou `--certificate-identity-regexp`. Sem um deles, o comando falha com `is required`

for verification in keyless mode. A expressão acima aceita qualquer workflow do repositório do Cilium.

Na saída, confirme três informações:

- **Issuer:** deve ser `https://token.actions.githubusercontent.com`;
- **Subject:** identifica o workflow e a tag que produziram a imagem, no formato `.../build-images-releases.yml@refs/tags/v1.7.0`;
- **docker-manifest-digest:** deve corresponder ao digest exibido pelo `docker pull` da mesma imagem.

8.2 Verificar a Assinatura do SBOM

Uma Lista de Materiais de Software (*Software Bill of Materials*, SBOM) é a relação formal e completa dos componentes usados para construir um software. Ela oferece visibilidade sobre a cadeia de suprimentos e sobre eventuais problemas de conformidade de licença e de segurança.

Desde a versão 0.8.4, todas as imagens do Tetragon incluem um SBOM, gerado no formato [SPDX](#) com a ferramenta [bom](#). Para garantir que o SBOM não foi adulterado, verifique sua assinatura:

```
cosign verify \  
  --certificate-identity-regexp "https://github.com/cilium/tetragon/.*" \  
  --certificate-github-workflow-repository cilium/tetragon \  
  --certificate-oidc-issuer https://token.actions.githubusercontent.com \  
  --attachment sbom quay.io/cilium/tetragon:v1.7.0 | jq
```

*Nota: os campos **Issuer** e **Subject** da saída permitem confirmar que a imagem foi assinada pelo GitHub Actions no repositório do Cilium.*

9 Configuração do Tetragon

O contêiner do Tetragon não guarda estado: toda a configuração é fornecida no momento em que ele é iniciado. Alterar uma opção significa, portanto, remover o contêiner e executá-lo novamente com os novos parâmetros, e não reconfigurar um agente em execução. A imagem, essa sim, é baixada uma única vez.

A configuração pode ser fornecida de três formas, equivalentes em resultado: arquivos de configuração no host, variáveis de ambiente ou argumentos na linha de comando. **Escolha uma delas**; não é necessário aplicar as três. Quando a mesma opção é definida em mais de um lugar, prevalece o argumento da linha de comando, seguido da variável de ambiente e, por último, do arquivo.

- **arquivos no host:** configuração versionável e comando de execução estável. É a forma adotada nesta documentação;
- **variáveis de ambiente:** adequada a Docker Compose e a pipelines de CI/CD;
- **argumentos na linha de comando:** adequada a testes pontuais, pois a configuração fica presa ao comando.

9.1 Configuração por Arquivos no Host

Nesta forma, cada opção corresponde a um arquivo em `/etc/tetragon/tetragon.conf.d/`, cujo nome é o da configuração e cujo conteúdo é o valor. O diretório é montado no contêiner, e o comando de execução permanece o mesmo, qualquer que seja a configuração.

1. Crie os diretórios necessários:

```
sudo mkdir -p /etc/tetragon/tetragon.conf.d /var/log/tetragon
```

2. Defina as opções desejadas:

O exemplo abaixo ativa a exportação dos eventos em JSON e a coleta das credenciais de processo (identificadores de usuário e grupo e capacidades associadas):

```
echo "/var/log/tetragon/tetragon.log" | \
  sudo tee /etc/tetragon/tetragon.conf.d/export-filename
echo "true" | \
  sudo tee /etc/tetragon/tetragon.conf.d/enable-process-cred
```

3. Inicie o contêiner montando o diretório de configuração:

```
sudo mkdir -p /etc/tetragon/tetragon.tp.d /var/log/tetragon

docker rm -f tetragon 2>/dev/null || true

docker run --name tetragon -d \
  --hostname "$(hostname)" \
  --pid=host --cgroupns=host --privileged \
  -v /sys/kernel:/sys/kernel \
  -v /etc/tetragon/tetragon.tp.d:/etc/tetragon/tetragon.tp.d:ro \
  -v /var/log/tetragon:/var/log/tetragon \
  quay.io/cilium/tetragon:v1.7.0 \
  /usr/bin/tetragon \
  --export-filename /var/log/tetragon/tetragon.log \
  --tracing-policy-dir /etc/tetragon/tetragon.tp.d
```

4. Verifique a configuração efetiva e a exportação:

```
docker logs tetragon | head -2 | grep -o "export-filename:[^ ]*"
sudo tail -f /var/log/tetragon/tetragon.log
```

Nota: a segunda linha do log de inicialização traz o mapa com todas as configurações efetivas, e é a forma mais direta de confirmar se uma opção foi lida. O `tail` permanece em execução até ser interrompido com `Ctrl + C`.

Nota: o volume `-v /var/log/tetragon:/var/log/tetragon` é obrigatório. Sem ele, o arquivo é gravado apenas dentro do contêiner e desaparece quando ele é removido.

Nota: sem nenhuma política de rastreamento carregada, o sensor base registra a execução e o encerramento de todos os processos da máquina, incluindo processos de kernel e serviços do sistema. O volume de eventos é elevado por natureza; é a aplicação de políticas, descrita nas seções seguintes, que restringe o registro ao que interessa.

9.2 Alternativas de Configuração

As duas formas a seguir produzem o mesmo resultado da anterior e são apresentadas apenas como referência. Não é necessário executá-las se a configuração por arquivos já foi aplicada.

Variáveis de ambiente:

Use o prefixo TETRAGON_ seguido do nome da configuração em maiúsculas, com hifens substituídos por sublinhados. Assim, --export-filename corresponde a TETRAGON_EXPORT_FILENAME.

```
docker rm -f tetragon 2>/dev/null || true

docker run --name tetragon -d \
  --hostname "$(hostname)" \
  --pid=host --cgroupns=host --privileged \
  --env "TETRAGON_EXPORT_FILENAME=/var/log/tetragon/tetragon.log" \
  -v /sys/kernel:/sys/kernel \
  -v /var/log/tetragon:/var/log/tetragon \
  -v /etc/tetragon/tetragon.tp.d:/etc/tetragon/tetragon.tp.d:ro \
  quay.io/cilium/tetragon:v1.7.0
```

Nota: a lista completa de argumentos e configurações está na [referência de configuração do daemon Tetragon](#).

10 Quick Local Docker Install

10.1 Iniciar o Tetragon

A maneira mais rápida de testar o Tetragon localmente é através do Docker.

1. Adicione o usuário atual ao grupo docker:

```
sudo usermod -aG docker $USER
```

2. Atualize a sessão para reconhecer as alterações de permissão:

```
newgrp docker
```

Nota: Caso necessário, encerre a conexão SSH atual (Ctrl + D) e reconecte-se:

```
ssh user@192.168.0.xxx
```

3. Execute o contêiner do Tetragon:

```
docker rm -f tetragon
docker run -d --name tetragon --rm \
  --pid=host --cgroupns=host --privileged \
  -v /sys/kernel/btf/vmlinux:/var/lib/tetragon/btf \
  quay.io/cilium/tetragon:v1.7.0
```

Nota: É mandatória a execução em modo privilegiado (--privileged) para permitir a carga e acoplamento dos programas BPF no kernel.

10.2 Executar o Aplicativo de Demonstração

Para testar a captura de eventos, implante o ambiente de demonstração com Docker Compose:

```
curl -LO \
  https://github.com/cilium/tetragon/raw/main/examples/quickstart/docker-compose.yaml
docker compose -f docker-compose.yaml up -d
```

Verifique se os contêineres foram inicializados corretamente:

```
docker container ls
```

11 Conceitos: Eventos e Políticas de Rastreamento

As seções anteriores aplicaram políticas prontas. Esta descreve como elas funcionam, de modo que seja possível escrever e adaptar políticas próprias em vez de apenas reutilizar exemplos.

11.1 Eventos

Todo evento do Tetragon descreve um processo e o que ele fez. Os principais tipos são:

- `process_exec`: execução de um binário;
- `process_exit`: encerramento de um processo, com o código de saída;
- `process_kprobe`: chamada de uma função do *kernel* monitorada por uma política;
- `process_tracepoint`: acionamento de um *tracepoint* monitorado por uma política;
- `process_uprobe`: chamada de uma função no espaço do usuário monitorada por uma política.

Os dois primeiros são produzidos pelo sensor base, sem nenhuma política carregada: é o motivo de o monitoramento de execução funcionar logo após a instalação. Os demais só existem quando há uma política que defina o ponto a ser observado.

Cada evento carrega o binário executado, os argumentos, o identificador do processo e do processo pai, as credenciais e o *cgroup* (que, em Docker, identifica o contêiner). A saída padrão é JSON; o modificador `-o compact` resume o evento em uma linha, como nas seções anteriores.

11.2 Anatomia de uma Política de Rastreamento

Uma política é um arquivo YAML com quatro blocos. O exemplo abaixo monitora leituras do arquivo `/tmp/segreto.txt`:

```

apiVersion: cilium.io/v1alpha1      # API usada pela policy
kind: TracingPolicy                  # tipo global (ver nota)
metadata:
  name: "monitorar-segredo"          # nome da policy
spec:
  kprobes:                           # ponto de observacao (hook point)
  - call: "security_file_permission"
    syscall: false
    args:                             # argumentos lidos da funcao
    - index: 0
      type: "file"                    # struct file: fornece o caminho
    - index: 1
      type: "int"                     # 0x04 = leitura, 0x02 = escrita
  selectors:                          # filtros aplicados no kernel
  - matchArgs:
    - index: 0
      operator: "Prefix"
      values:
      - "/tmp/segredo.txt"
    - index: 1
      operator: "Equal"
      values:
      - "4"

```

Nota: existe também o tipo `TracingPolicyNamespaced`, que exige o campo `metadata.namespace` e depende do Kubernetes. Em ambientes Docker, utilize sempre `TracingPolicy`.

11.3 Pontos de Observação

O bloco `spec` define onde a política se conecta ao *kernel*:

- **kprobes**: funções internas do *kernel*. Com `syscall: true`, o nome da chamada de sistema é resolvido para o símbolo específico da arquitetura (por exemplo, `sys_execve` para `__x64_sys_execve`);
- **tracepoints**: pontos de instrumentação estáveis, definidos por subsistema e evento;
- **uprobes** e **usdt**: funções em binários e bibliotecas do espaço do usuário;
- **lsm**: ganchos do Linux Security Module, disponíveis quando o *kernel* tem BPF LSM habilitado;
- **enforcers**: declaração auxiliar usada por políticas de aplicação em *kernels* sem suporte a *kprobe multi*.

Nota: as funções que começam com `security_` são ganchos de segurança do kernel e costumam ser preferíveis às chamadas de sistema equivalentes, por serem acionadas após a resolução de caminhos e não estarem sujeitas a condições de corrida.

11.4 Argumentos

O bloco `args` informa ao Tetragon como interpretar cada parâmetro da função observada, pelo índice e pelo tipo. Os tipos mais comuns são `int`, `uint32`, `string`, `file`, `path`, `sock` e `linux_binprm`. O tipo determina o que os filtros conseguem comparar: com `file`, por exemplo, os operadores atuam sobre o caminho do arquivo.

O campo `returnArg` captura o valor de retorno da função, e exige `return: true` na definição do gancho. É o que permite registrar se uma operação foi bem-sucedida ou negada.

11.5 Seletores

Os seletores são os filtros aplicados dentro do *kernel*, e por isso definem tanto o que será registrado quanto a sobrecarga da política:

- `matchArgs`: compara os argumentos da chamada;
- `matchReturnArgs`: compara o valor de retorno;
- `matchBinaries` e `matchParentBinaries`: restringem a política a determinados executáveis ou a seus processos pais;
- `matchPIDs`: filtra por identificadores de processo, com as opções `followForks` e `isNamespacePID`;
- `matchCapabilities`: filtra pelas capacidades do processo;
- `matchActions`: define a ação executada quando o seletor casa.

Os operadores variam conforme o tipo do argumento. Para textos e caminhos: `Equal`, `NotEqual`, `Prefix`, `Postfix`, `In` e `NotIn`. Para valores numéricos e sinalizadores: `Mask` e `InMap`. Para soquetes: `DAddr`, `NotDAddr`, `DPort` e `SPort`, usados na política de rede da seção anterior.

Nota sobre a lógica de combinação: dentro de um mesmo seletor, todos os blocos precisam casar simultaneamente (E lógico); já os seletores de uma mesma lista são avaliados de forma independente (OU lógico). É por isso que a política de arquivos repete a estrutura `matchArgs` três vezes: uma para leituras de arquivos sensíveis, outra para leituras de arquivos de configuração de shell e outra para escritas em diretórios sensíveis.

11.6 Ações

As ações são declaradas em `matchActions` e determinam o que acontece quando o filtro casa:

- `Post`: envia o evento ao espaço do usuário. É o comportamento padrão das políticas de observação;
- `NoPost`: suprime o evento, útil para reduzir ruído;
- `Sigkill`: encerra imediatamente o processo que fez a chamada;
- `Signal`: envia um sinal arbitrário, informado no campo `argSig`;

- **Override**: faz a chamada retornar um erro definido em `argError`, sem encerrar o processo;
- **NotifyEnforcer**: aciona o programa declarado em `enforcers`;
- **TrackSock** e **UntrackSock**: mantêm a associação entre soquetes e processos.

Nota: `Post`, `Sigkill`, `Override`, `TrackSock` e `UntrackSock` são executados no próprio kernel, o que garante que a ação ocorra antes da conclusão da operação observada. Já `GetUrl` e `DnsLookup` acontecem no espaço do usuário, após o recebimento do evento.

11.7 Modo de Aplicação

Uma política com ações de aplicação pode ser carregada sem que essas ações sejam executadas, o que é útil para avaliar o impacto antes de bloquear qualquer coisa. Os modos são:

- **monitoring**: as ações de aplicação são ignoradas;
- **enforcement**: as ações são executadas;
- **monitor_only**: a política não possui ações de aplicação.

O modo pode ser definido no próprio arquivo, pelo campo `spec.options`:

```
spec:
  options:
    - name: "policy-mode"
      value: "monitor"
```

E consultado ou alterado pela CLI, com prioridade sobre o que está no arquivo:

```
docker exec tetragon tetra tracingpolicy list
```

11.8 Exemplo Prático

O procedimento a seguir aplica a política mostrada no início desta seção, que apenas observa, sem encerrar processos.

1. Crie o arquivo de teste e a política:

Escreva o arquivo da política com um *here-document*, copiando o bloco inteiro, da linha `cat` até o EOF final, e colando de uma só vez. Editores de texto em terminal são uma fonte frequente de arquivos salvos vazios, o que faz o Tetragon encerrar ao iniciar.

```

echo "conteudo sensivel" > /tmp/segredo.txt
cat > monitorar_segredo.yaml <<'EOF'
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "monitorar-segredo"
spec:
  kprobes:
  - call: "security_file_permission"
    syscall: false
    args:
    - index: 0
      type: "file"
    - index: 1
      type: "int"
  selectors:
  - matchArgs:
    - index: 0
      operator: "Prefix"
      values:
      - "/tmp/segredo.txt"
    - index: 1
      operator: "Equal"
      values:
      - "4"
EOF
ls -l monitorar_segredo.yaml

```

Nota: o ls confirma que o arquivo não está vazio. Um arquivo de 0 byte é aceito pelo Docker como volume, mas o agente encerra ao tentar carregá-lo.

2. Inicie o Tetragon com a política montada:

```

mkdir -p politicas
# crie politicas/monitorar_segredo.yaml com o here-document

docker run -d --name tetragon \
  --hostname "$(hostname)" \
  --pid=host --cgroupns=host --privileged \
  -v "$PWD/politicas":/etc/tetragon/tetragon.tp.d:ro \
  -v /sys/kernel/btf/vmlinux:/var/lib/tetragon/btf:ro \
  quay.io/cilium/tetragon:v1.7.0

```

3. Confirme que a política foi carregada:

```
docker exec tetragon tetra tracingpolicy list
```

4. Observe os eventos (Terminal 01) e gere uma leitura (Terminal 02):

```
docker exec -ti tetragon tetra getevents -o compact
```

```
cat /tmp/segredo.txt
```

Nota: o Tetragon só registra o que ocorre após o carregamento da política. Leituras feitas antes de o contêiner subir não aparecem, e o contador NPOST do tracingpolicy list permanece em zero até o primeiro evento.

Nota: se o contêiner encerrar logo após iniciar, execute o mesmo comando sem `-d` e sem `--rm` para ler a mensagem de erro. Em caso de sucesso, as últimas linhas do log são `Added TracingPolicy with success` e `Listening for events....`

12 Monitoramento de Execução

O Tetragon rastreia execuções em clusters Kubernetes, máquinas virtuais ou ambientes *bare metal*, correlacionando os binários invocados aos metadados correspondentes (contêiner, Pod, nó e cluster).

12.1 Observar Eventos de Execução

Os eventos são disponibilizados via logs em JSON ou por fluxo gRPC.

Terminal 01 (Monitoramento de Eventos):

```
docker exec tetragon tetra getevents -o compact
```

Terminal 02 (Disparo de Ações de Teste):

```
docker exec -ti starwars-xwing-1 curl https://ebpf.io/applications/#tetragon
```

A CLI exibirá um log compacto similar a:

```
process default/xwing /bin/bash -c "curl https://ebpf.io/applications/#tetragon"
process default/xwing /usr/bin/curl https://ebpf.io/applications/#tetragon
exit    default/xwing /usr/bin/curl https://ebpf.io/applications/#tetragon 60
```

O formato compacto resume: tipo do evento, identificador do *pod*, binário executado, parâmetros e o código de retorno (*exit code* 60, no exemplo).

Visualização Completa em JSON:

Para inspecionar metadados detalhados (como identificadores de namespaces, contêiner e labels do Kubernetes), omita o modificador de saída:

```
docker exec -ti tetragon tetra getevents
```

13 Monitoramento de Acesso a Arquivos

Políticas de rastreamento (*tracing policies*) podem ser adicionadas ao Tetragon por meio de arquivos de configuração YAML, que estendem o rastreamento de execução padrão. Essas políticas realizam a filtragem diretamente no *kernel*, garantindo que apenas os eventos relevantes sejam enviados ao espaço do usuário pelos programas BPF. Assim, a sobrecarga permanece baixa mesmo em sistemas com alta carga.

A política utilizada nesta seção é a [file_monitoring.yaml](#), que pode ser revisada e estendida conforme a necessidade. Os arquivos monitorados por ela formam um bom conjunto inicial.

13.1 Aplicar a Política de Rastreamento

*Nota: estas instruções pressupõem que o aplicativo de demonstração já foi implantado, conforme descrito na seção *Quick Local Docker Install*.*

1. Baixe o arquivo da política:

```
wget \
https://github.com/cilium/tetragon/raw/main/examples/quickstart/file_monitoring.yaml
```

2. Reinicie o Tetragon montando a política no diretório de políticas do contêiner:

O `docker rm -f` remove o contêiner em execução e libera o nome `tetragon`; sem ele, o Docker recusa a nova execução.

```
docker rm -f tetragon
docker run -d --name tetragon --rm \
  --pid=host --cgroupns=host --privileged \
  -v ${PWD}/file_monitoring.yaml:/etc/tetragon/tetragon.tp.d/file_monitoring.yaml \
  -v /sys/kernel/btf/vmlinux:/var/lib/tetragon/btf \
  quay.io/cilium/tetragon:v1.7.0
```

Nota: `${PWD}` corresponde ao diretório atual. Execute o comando na mesma pasta em que o arquivo `file_monitoring.yaml` foi baixado.

13.2 Observar Eventos de Acesso a Arquivos

Com a política aplicada, utilize novamente a CLI `tetra` para acompanhar os eventos.

Terminal 01 (Monitoramento de Eventos):

```
docker exec -ti tetragon tetra getevents -o compact
```

Terminal 02 (Leitura de Arquivo Sensível):

Para gerar um evento, leia um arquivo sensível referenciado na política:

```
docker exec -ti starwars-xwing-1 bash -c "cat /etc/shadow"
```

A CLI exibirá um evento de leitura similar a:

```
process default/xwing /bin/bash -c "cat /etc/shadow"
process default/xwing /bin/cat /etc/shadow
read    default/xwing /bin/cat /etc/shadow
exit    default/xwing /bin/cat /etc/shadow 0
```

Terminal 02 (Gravação em Diretório Sensível):

A política também gera eventos de gravação quando há tentativas de escrita em diretórios sensíveis, como o `/etc`:

```
docker exec -ti starwars-xwing-1 bash -c "echo foo >> /etc/bar"
```

A saída será semelhante a:

```
process default/xwing /bin/bash -c "echo foo >> /etc/bar"
write  default/xwing /bin/bash /etc/bar
write  default/xwing /bin/bash /etc/bar
exit   default/xwing /bin/bash -c "echo foo >> /etc/bar"
```

Nota: os exemplos de saída vêm de um ambiente Kubernetes. No Docker, os metadados do Kubernetes (como `default/xwing`) não aparecem, e a CLI também exibe ícones antes de cada tipo de evento, omitidos aqui.

14 Monitoramento de Rede

Nesta seção, o objetivo é monitorar o tráfego de rede destinado a endereços externos, ou seja, fora do cluster Kubernetes ou externos ao host Docker onde o Tetragon é executado. As instruções pressupõem que o Tetragon já esteja em execução e que o aplicativo de demonstração do Cilium tenha sido implantado.

14.1 Monitoramento de Rede em Docker ou Bare Metal

Como o Docker não possui os conceitos de CIDR de *pod* ou CIDR de serviço, é necessário criar uma política de rastreamento que filtre o endereço de *loopback* (127.0.0.1) do log de eventos do Tetragon.

1. Defina as variáveis de ambiente com o endereço de loopback:

```
export PODCIDR="127.0.0.1/32"
export SERVICECIDR="127.0.0.1/32"
```

2. Baixe a política e substitua as variáveis com o envsubst:

```
wget \
  https://github.com/cilium/tetragon/raw/main/examples/quickstart/network_egress_cluster.yaml
envsubst < network_egress_cluster.yaml > network_egress_cluster_subst.yaml
```

Nota: o envsubst faz parte do pacote gettext-base. Caso o comando não seja encontrado, instale-o com `sudo apt install gettext-base`.

Nota: o arquivo `network_egress_cluster.yaml` é do tipo `TracingPolicyNamespaced`, que exige o campo `metadata.namespace` e só funciona no Kubernetes. Em Docker, o Tetragon encerra com o erro `validation failed: metadata.namespace: Required value`. Antes de montar a política, troque o tipo para `TracingPolicy`, que é global:

```
sed -i 's/kind: TracingPolicyNamespaced/kind: TracingPolicy/' \
  network_egress_cluster_subst.yaml
```

3. Reinicie o Tetragon com a nova política:

```
docker rm -f tetragon
docker run -d --name tetragon --rm \
  --pid=host --cgroupns=host --privileged \
  -v ${PWD}/network_egress_cluster_subst.yaml:/etc/tetragon/tetragon.tp.d/network_egress_cluster_subst.yaml \
  -v /sys/kernel/btf/vmlinux:/var/lib/tetragon/btf \
  quay.io/cilium/tetragon:v1.7.0
```

14.2 Observar Eventos de Conexão

Com a política aplicada, as conexões TCP remotas passam a ser registradas, enquanto as conexões com o endereço local são filtradas pelo Tetragon.

Terminal 01 (Monitoramento de Eventos):

```
docker exec -ti tetragon tetra getevents -o compact
```

Terminal 02 (Geração de uma Conexão Remota):

```
curl https://ebpf.io/applications/
```

A saída será semelhante a:

```
process /usr/bin/curl https://ebpf.io/applications/
connect /usr/bin/curl tcp 192.168.1.190:36124 -> 104.198.14.52:443
exit    /usr/bin/curl https://ebpf.io/applications/ 0
```

Nota: o endereço de origem corresponde ao IP do host e o de destino ao servidor acessado; ambos variam conforme o ambiente.

15 Aplicação de Políticas

Além de monitorar funções do *kernel* para relatar eventos de acesso a arquivos ou de conexões de rede, as políticas de rastreamento do Tetragon podem impor restrições sobre essas mesmas funções. Como a filtragem ocorre no *kernel*, há ganho de desempenho, pois limita-se o volume de eventos enviados ao espaço do usuário, e também se torna possível aplicar as restrições no nível do *kernel*.

Nesta seção, a aplicação de políticas é somada às funcionalidades já implementadas (políticas de execução, rastreamento de arquivos e rastreamento de rede). Especificamente, serão abordados dois casos:

- restringir o tráfego de rede que sai do ambiente;
- bloquear operações de leitura e gravação em arquivos sensíveis.

15.1 Impor Restrições de Acesso a Arquivos

O exemplo a seguir amplia o monitoramento de acesso a arquivos com a aplicação de regras, garantindo que arquivos sensíveis não sejam lidos. A política utilizada é a [file_monitoring_enforce.yaml](#). A única diferença em relação à política de observação é a inclusão de um bloco de ação que envia um SIGKILL ao processo e retorna um erro na operação.

1. Baixe a política:

```
wget \
https://github.com/cilium/tetragon/raw/main/examples/quickstart/file_monitoring_enforce.yaml
```

2. Converta o tipo da política antes de utilizá-la:

Como no monitoramento de rede, este arquivo é do tipo `TracingPolicyNamespaced`. Sem a conversão, o Tetragon encerra ao iniciar.

```
sed -i 's/kind: TracingPolicyNamespaced/kind: TracingPolicy/' \
file_monitoring_enforce.yaml
head -4 file_monitoring_enforce.yaml
```

3. Reinicie o Tetragon com a política aplicada:

```
docker rm -f tetragon
docker run -d --name tetragon --rm \
--pid=host --cgroupns=host --privileged \
-v ${PWD}/file_monitoring_enforce.yaml:/etc/tetragon/tetragon.tp.d/file_monitoring_enforce.yaml \
-v /sys/kernel/btf/vmlinux:/var/lib/tetragon/btf \
quay.io/cilium/tetragon:v1.7.0
```

Aviso: convertida para `TracingPolicy`, a política passa a valer para toda a máquina, e não apenas para o contêiner de demonstração. Ela encerra com `SIGKILL` qualquer processo que leia arquivos como `/etc/shadow` ou escreva em `/etc`, o que pode atingir gerenciadores

de pacotes, editores e serviços de log. Faça o teste e, em seguida, pare o contêiner com `docker rm -f tetragon`.

4. Acompanhe os eventos (Terminal 01):

```
docker exec -ti tetragon tetra getevents -o compact
```

5. Tente ler um arquivo sensível listado na política (Terminal 02):

```
docker exec -ti starwars-xwing-1 bash -c "cat /etc/shadow"
```

Como o arquivo está incluído na política, o processo é encerrado e o comando falha:
`command terminated with exit code 137`

*Nota: o código 137 indica encerramento por **SIGKILL** ($128 + 9$), ou seja, a ação de aplicação da política foi executada. Em paralelo, o Terminal 01 exibe o evento de leitura correspondente.*

16 Monitoramento de Escape de Contêiner

Um escape de contêiner ocorre quando um processo em execução dentro de um contêiner obtém acesso ao host ou a outros contêineres. Como o isolamento em Linux é feito por *namespaces*, *cgroups* e capacidades, e não por uma fronteira de máquina virtual, o escape quase sempre se manifesta como uma alteração nesses mecanismos ou como o uso de um recurso do host indevidamente exposto ao contêiner.

16.1 Indicadores Observáveis

Os comportamentos a seguir são os mais úteis para detecção, pois ocorrem em praticamente todas as técnicas conhecidas:

- **mudança de *namespace***: uso de `setns` ou `unshare`, típico de ferramentas como o `nsenter` e da criação de *namespaces* de usuário para obter privilégios;
- **montagem de sistemas de arquivos**: `mount` dentro do contêiner, frequentemente usado para expor o disco do host;
- **acesso ao soquete do Docker**: leitura ou escrita em `/var/run/docker.sock`, que permite criar contêineres privilegiados e é equivalente a acesso administrativo ao host;
- **escrita em arquivos de controle do kernel**: `/proc/sys/kernel/core_pattern` e o `release_agent` de *cgroups*, ambos usados em técnicas clássicas de escape;
- **ganho de capacidades**: aquisição de `CAP_SYS_ADMIN` ou `CAP_SYS_MODULE` por um processo que não as possuía.

16.2 Pré-requisitos

A visibilidade sobre *namespaces* e credenciais não vem habilitada por padrão e é necessária para os dois últimos indicadores:

```
echo "true" | sudo tee /etc/tetragon/tetragon.conf.d/enable-process-ns  
echo "true" | sudo tee /etc/tetragon/tetragon.conf.d/enable-process-cred
```

Nota: com essas opções, os eventos passam a incluir os identificadores de namespace e as capacidades do processo, o que permite comparar o estado no momento da execução com o estado atual.

16.3 Política de Detecção

A política a seguir cobre os quatro primeiros indicadores em modo de observação. Crie o diretório de políticas e o arquivo:

```

mkdir -p politicas
cat > politicas/escape_container.yaml <<'EOF'
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "detectar-escape-container"
spec:
  kprobes:
    # 1. Mudanca de namespace (nsenter, setns)
    - call: "sys_setns"
      syscall: true
      args:
        - index: 0
          type: "int"
        - index: 1
          type: "int"

    # 2. Criacao de namespaces (unshare)
    - call: "sys_unshare"
      syscall: true
      args:
        - index: 0
          type: "int"

    # 3. Montagem de sistemas de arquivos
    - call: "sys_mount"
      syscall: true
      args:
        - index: 0
          type: "string"
        - index: 1
          type: "string"
        - index: 2
          type: "string"

    # 4. Acesso a recursos sensiveis do host
    - call: "security_file_permission"
      syscall: false
      args:
        - index: 0
          type: "file"
        - index: 1
          type: "int"
      selectors:
        - matchArgs:
            - index: 0
              operator: "Prefix"
              values:
                - "/var/run/docker.sock"
                - "/run/docker.sock"
                - "/proc/sys/kernel/core_pattern"
                - "/sys/fs/cgroup"
EOF

```

16.4 Aplicação e Teste

1. Inicie o agente com o diretório de políticas:

```
docker rm -f tetragon
docker run -d --name tetragon --rm --hostname $(hostname) \
  --pid=host --cgroupns=host --privileged \
  -v /etc/tetragon:/etc/tetragon \
  -v /sys/kernel:/sys/kernel \
  -v ${PWD}/politicas:/etc/tetragon/tetragon.tp.d \
  quay.io/cilium/tetragon:v1.7.0 \
  /usr/bin/tetragon
```

2. Acompanhe os eventos (Terminal 01):

```
docker exec -ti tetragon tetra getevents -o compact
```

3. Simule os comportamentos (Terminal 02):

```
sudo unshare --mount /bin/true
sudo nsenter -t 1 -m /bin/true
```

4. Simule o acesso ao soquete do Docker a partir de um contêiner:

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  curlimages/curl:latest \
  -s --unix-socket /var/run/docker.sock http://localhost/version
```

Nota: este último comando reproduz, de forma controlada, o cenário mais comum de escape em ambientes Docker: um contêiner com o soquete do daemon montado. O evento gerado identifica o processo e o caminho acessado.

16.5 Contenção

Uma vez validada a detecção e ajustados os falsos positivos, a política pode passar a bloquear. Acrescente o bloco de ação ao seletor desejado:

```
selectors:
- matchArgs:
  - index: 0
    operator: "Prefix"
    values:
  - "/var/run/docker.sock"
matchActions:
- action: Override
  argError: -1
```

*Nota: a ação **Override** faz a operação falhar com o erro indicado, sem encerrar o processo, e exige **CONFIG_BPF_KPROBE_OVERRIDE** no kernel, verificável com **tetra probe config**. A alternativa **Sigkill** encerra o processo imediatamente e deve ser reservada a comportamentos sem qualquer uso legítimo no ambiente.*

Aviso: antes de habilitar a contenção, avalie o impacto. Ferramentas legítimas de orquestração e de observabilidade também acessam o soquete do Docker e realizam montagens; uma política ampla demais pode interromper a operação normal do host.

16.6 Limitações

O Tetragon observa o comportamento do processo, não a intenção. Escapes que exploram falhas do próprio *kernel*, sem passar por chamadas de sistema monitoradas, podem não gerar

nenhum dos eventos acima. Da mesma forma, um contêiner iniciado com `--privileged` ou com `--pid=host` já possui acesso ao host por configuração, sem necessidade de escape: nesse caso, a detecção relevante ocorre na criação do contêiner, e não em tempo de execução.

17 Métricas

O Tetragon expõe métricas no formato Prometheus, utilizadas com dois propósitos principais:

- monitorar a saúde do próprio Tetragon;
- acompanhar a atividade dos processos observados por ele.

A lista completa está na [referência de métricas](#).

17.1 Habilitar as Métricas fora do Kubernetes

Em instalações fora do Kubernetes, as métricas vêm desabilitadas por padrão. Para habilitá-las, defina o endereço do servidor de métricas por meio do argumento `--metrics-server` (por exemplo, `:2112`). Em instalações via pacote com `systemd`, defina a entrada `metrics-address` em um arquivo dentro do diretório `/etc/tetragon/tetragon.conf.d/`.

1. Inicie o contêiner com as métricas habilitadas:

```
docker rm -f tetragon
docker run -d --name tetragon --rm --privileged \
  --pid=host --cgroupns=host \
  -p 2112:2112 \
  -v /sys/kernel/btf/vmlinux:/var/lib/tetragon/btf \
  quay.io/cilium/tetragon:v1.7.0 \
  /usr/bin/tetragon --metrics-server :2112
```

2. Verifique se as métricas estão expostas:

```
curl http://localhost:2112/metrics
```

17.2 Configurar Rótulos nas Métricas de Eventos

As métricas de eventos são divididas por rótulos, e cada combinação de valores gera uma série temporal distinta no Prometheus. Em ambientes com muitas cargas de trabalho, isso produz cardinalidade elevada. O conjunto de rótulos preenchidos é definido pela configuração `metrics-label-filter`, que recebe os rótulos habilitados separados por vírgula.

O valor padrão é `namespace,workload,pod,binary`. Os três primeiros dependem da API do Kubernetes e permanecem vazios em instalações Docker, de modo que, neste ambiente, o único rótulo aplicável é `binary`, que separa as métricas por binário de origem.

Pela configuração em arquivo, forma adotada nesta documentação:

```
echo "binary" | \
  sudo tee /etc/tetragon/tetragon.conf.d/metrics-label-filter
```

Ou como argumento, ao final do docker run:

```
docker rm -f tetragon
docker run -d --name tetragon --rm --hostname $(hostname) \
  --pid=host --cgroupns=host --privileged \
  -p 2112:2112 \
  -v /sys/kernel/btf/vmlinux:/var/lib/tetragon/btf \
  quay.io/cilium/tetragon:v1.7.0 \
  /usr/bin/tetragon --metrics-server :2112 --metrics-label-filter "binary"
```

Nota: em instalações no Kubernetes, o mesmo ajuste é feito pelo valor `tetragon.prometheus.metrics` do gráfico Helm. Não se aplica a este ambiente.

17.3 Coleta das Métricas com o Prometheus

Fora do Kubernetes não existe o recurso *ServiceMonitor*, então o alvo de coleta é declarado diretamente no arquivo `prometheus.yml`. O procedimento a seguir sobe o Tetragon e o Prometheus com o Docker Compose, o que coloca ambos na mesma rede e permite que o Prometheus alcance o agente pelo nome do serviço.

Nota: o nome `host.docker.internal`, comum em exemplos da Internet, só existe no Docker Desktop. No Docker Engine em Linux ele não é resolvido sem configuração adicional, motivo pelo qual esta seção utiliza o Compose.

1. Crie um diretório para o conjunto e entre nele:

```
mkdir -p ~/tetragon-metrics && cd ~/tetragon-metrics
```

2. Crie o arquivo `prometheus.yml`:

```
cat > prometheus.yml <<'EOF'
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'tetragon'
    scrape_interval: 10s
    static_configs:
      - targets: ['tetragon:2112']
EOF
```

3. Crie o arquivo `docker-compose.yml`:

```
cat > docker-compose.yaml <<'EOF'
services:
  tetragon:
    image: quay.io/cilium/tetragon:v1.7.0
    container_name: tetragon
    privileged: true
    pid: "host"
    cgroup: "host"
    volumes:
      - /sys/kernel/btf/vmlinux:/var/lib/tetragon/btf
    command: /usr/bin/tetragon --metrics-server :2112
    ports:
      - "2112:2112"

  prometheus:
    image: prom/prometheus:v3.6.0
    container_name: prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
EOF
```

4. Remova o contêiner do Tetragon em execução e suba o conjunto:

O Compose cria o seu próprio contêiner com o nome `tetragon`, que entra em conflito com o que já estiver ativo.

```
docker rm -f tetragon
docker compose up -d
docker compose ps
```

5. Verifique se o Tetragon está expondo as métricas:

```
curl -s http://localhost:2112/metrics | head -20
```

6. Verifique se o Prometheus está coletando:

```
curl -s http://localhost:9090/api/v1/targets | grep -o '"health":"[a-z]*"'
```

A resposta esperada é `"health":"up"`. A interface gráfica fica disponível em `http://<IP-da-VM>:9090`, na aba *Status* e em seguida *Targets*.

7. Consulte uma métrica:

```
curl -s "http://localhost:9090/api/v1/query?query=tetragon_events_total" | head -5
```

8. Para encerrar o conjunto:

```
docker compose down
```

Nota: o Tetragon subido pelo Compose não carrega as políticas nem a configuração de `/etc/tetragon`. Para monitorar com políticas e métricas ao mesmo tempo, acrescente ao serviço `tetragon` os volumes correspondentes, como `/etc/tetragon:/etc/tetragon` e o diretório das políticas.

18 Operação do Ambiente

Esta seção descreve o ambiente já instalado como um todo: onde cada configuração reside, como iniciar, reiniciar e atualizar os componentes, e o que alterar para monitorar ou conter uma ameaça específica.

18.1 Visão Geral do Fluxo

A Figura 1 resume o caminho percorrido por um evento, desde a chamada de função no *kernel* até a saída consumida pelo analista.

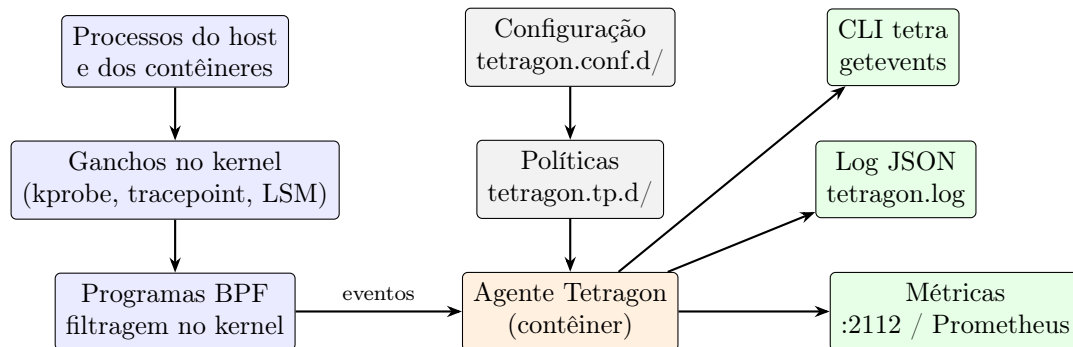


Figura 1: Fluxo de eventos e pontos de configuração do ambiente.

Dois pontos merecem destaque. Primeiro, a filtragem ocorre no *kernel*: sem políticas, apenas execução e encerramento de processos são registrados, e o volume é alto. Segundo, o agente não guarda estado: as políticas e a configuração vêm de diretórios do host montados no contêiner, de modo que qualquer alteração exige recriá-lo.

18.2 Onde Fica Cada Coisa

- `/etc/tetragon/tetragon.conf.d/`: configuração do agente. Um arquivo por opção, com o nome da opção e o valor como conteúdo;
- `/etc/tetragon/tetragon.tp.d/`: políticas de rastreamento, em YAML. Este diretório fica dentro do contêiner e recebe as políticas por montagem de volume;
- `/var/log/tetragon/tetragon.log`: eventos exportados em JSON, quando `export-filename` está definido;
- imagem `quay.io/cilium/tetragon:v1.7.0`: o agente em si, armazenado localmente após o primeiro download;
- `/usr/local/bin/tetra`: a CLI, instalada no host.

18.3 Operações do Dia a Dia

Iniciar o ambiente com configuração e políticas:

```
docker rm -f tetragon
docker run -d --name tetragon --rm --hostname $(hostname) \
  --pid=host --cgroupns=host --privileged \
  -v /etc/tetragon:/etc/tetragon \
  -v /sys/kernel:/sys/kernel \
  -v /var/log/tetragon:/var/log/tetragon \
  -v ${PWD}/politicass:/etc/tetragon/tetragon.tp.d \
  quay.io/cilium/tetragon:v1.7.0 \
  /usr/bin/tetragon
```

Nota: montar um diretório inteiro de políticas, em vez de arquivo por arquivo, permite acrescentar ou remover políticas sem alterar o comando. Crie o diretório com `mkdir -p politicass` e coloque nele os arquivos YAML desejados.

Verificar o estado:

```
docker ps --filter name=tetragon
docker exec tetragon tetra --timeout 60s tracingpolicy list
docker logs tetragon | head -2
```

Acompanhar eventos:

```
docker exec -ti tetragon tetra getevents -o compact
```

Acrescentar ou alterar uma política:

```
# 1. edite ou adicione o arquivo YAML em ./politicass
# 2. recrie o agente para recarregar o diretório
docker rm -f tetragon
# 3. repita o comando de inicializacao acima
```

Alterar uma configuração:

```
echo "true" | sudo tee /etc/tetragon/tetragon.conf.d/enable-process-cred
docker rm -f tetragon
# repita o comando de inicializacao
```

Atualizar o Tetragon:

```
docker pull quay.io/cilium/tetragon:v1.8.0
docker rm -f tetragon
# repita o comando de inicializacao com a nova tag
```

Nota: substitua v1.8.0 pela versão desejada, conforme as tags publicadas no Quay. Após atualizar o agente, atualize também a CLI, para manter as versões alinhadas, e repita a verificação de assinatura descrita na seção Verificação da Instalação.

Encerrar o ambiente:

```
docker rm -f tetragon
```

Nota: ao encerrar o agente, os programas BPF são descarregados e o monitoramento e a contenção deixam de valer. Para que as políticas de aplicação permaneçam ativas mesmo com o agente fora do ar, consulte a configuração `keep-sensors-on-exit` na documentação oficial.

18.4 Resposta a uma Ameaça Identificada

O procedimento típico, partindo de um comportamento suspeito observado nos eventos, é:

1. identificar o gancho apropriado, normalmente uma função `security_*` para arquivos, ou a chamada de sistema correspondente;
2. escrever a política em modo de observação, com a ação `Post`, e coletar eventos por tempo suficiente para avaliar falsos positivos;
3. restringir os seletores até que apenas o comportamento indesejado seja registrado;
4. acrescentar a ação de contenção, `Sigkill` para encerrar o processo ou `Override` para fazer a operação falhar, mantendo o modo `monitoring` enquanto valida;
5. alterar o modo para `enforcement` e recriar o agente.

Nota: prefira `Override` a `Sigkill` em máquinas em uso. A operação falha com o erro definido em `argError`, sem encerrar o processo, o que reduz o impacto em caso de política mal calibrada. O suporte é verificável com `tetra probe config`, na opção `CONFIG_BPF_KPROBE_OVERRIDE`.

19 Troubleshooting

Esta seção reúne os procedimentos de diagnóstico mais frequentes na operação do Tetragon em contêiner, na ordem em que costumam ser necessários.

19.1 O Contêiner Sobe e Encerra em Seguida

Este é o sintoma mais comum. O `docker run` exibe o identificador do contêiner, mas o `docker ps` não o lista. Como o contêiner é iniciado com `--rm`, ele é removido ao encerrar e não aparece nem em `docker ps -a`.

Execute o mesmo comando em primeiro plano, sem `-d` e sem `--rm`, para que a mensagem de erro permaneça na tela:

```
docker run --name tetragon-debug \
  --pid=host --cgroupns=host --privileged \
  -v ${PWD}/politica.yaml:/etc/tetragon/tetragon.tp.d/politica.yaml \
  -v /sys/kernel/btf/vmlinux:/var/lib/tetragon/btf \
  quay.io/cilium/tetragon:v1.7.0
```

A causa aparece nas últimas linhas do log. As mais frequentes são:

- `validation failed: metadata.namespace: Required value:` a política é do tipo `TracingPolicyNamespaced`. Converta para `TracingPolicy`, como descrito nas seções de monitoramento de rede e de aplicação de políticas;
- erro de leitura ou de sintaxe do YAML: o arquivo não foi baixado corretamente. Confirme o conteúdo com `head -3 politica.yaml`; se aparecer HTML, o download trouxe uma página de erro;
- ausência de `/sys/kernel/btf/vmlinux`: o *kernel* não expõe BTf, requisito do Tetragon.

Nota: ao terminar, remova o contêiner de diagnóstico com `docker rm -f tetragon-debug`, pois sem `--rm` ele permanece e ocupa o nome.

19.2 Verificar o Estado e a Configuração Efetiva

```
docker ps --filter name=tetragon
docker logs tetragon | head -3
docker exec tetragon ls /etc/tetragon/tetragon.tp.d/
docker exec tetragon tetra tracingpolicy list
```

A segunda linha do log traz o mapa com todas as configurações efetivas, o que permite confirmar se um parâmetro definido por arquivo ou por variável de ambiente foi realmente aplicado. O `tracingpolicy list` mostra as políticas carregadas, o estado, o modo e a memória de *kernel* consumida por cada uma.

19.3 Tempo Limite Excedido na CLI

Alguns subcomandos da CLI utilizam um tempo limite curto e podem falhar com a mensagem `rpc error: code = DeadlineExceeded`, mesmo com o agente em execução. O caso mais comum é o `tracingpolicy list` em ambientes com muitos eventos.

Aumente o tempo limite pelo sinalizador global `--timeout`, que deve vir antes do subcomando:

```
docker exec tetragon tetra --timeout 60s tracingpolicy list
```

A saída confirma, para cada política: o estado, o escopo, o sensor utilizado, a memória de *kernel* consumida, o modo de aplicação e os contadores de eventos postados (NPOST), bloqueios (NENFORCE) e ocorrências em modo de monitoramento (NMONITOR).

Nota: se o comando continuar falhando, a política pode ser verificada pelo diretório montado, com `docker exec tetragon ls /etc/tetragon/tetragon.tp.d/`, e pelas linhas de carregamento do sensor em `docker logs tetragon`.

19.4 Verificar os Requisitos do Kernel

Quando um gancho não carrega ou um recurso não funciona, verifique as opções de compilação e as capacidades do *kernel* em uso:

```
docker exec tetragon tetra probe config
docker exec tetragon tetra probe
```

O primeiro comando lista as opções de configuração relevantes, como `CONFIG_BPF`, `CONFIG_DEBUG_INFO_B` e `CONFIG_BPF_KPROBE_OVERRIDE`, esta última necessária para a ação `Override`. O segundo detecta em tempo de execução os recursos disponíveis, como *kprobe multi* e BPF LSM.

19.5 Nível de Log

O nível padrão é `info`. Para diagnósticos mais detalhados, defina `debug` pela configuração: `echo "debug" | sudo tee /etc/tetragon/tetragon.conf.d/log-level`

O nível também pode ser alterado sem reiniciar o contêiner, pelo subcomando `loglevel`:

```
docker exec tetragon tetra loglevel get
docker exec tetragon tetra loglevel set debug
```

Nota: o nível `debug` gera volume elevado de mensagens; use apenas durante a investigação e retorne a `info` em seguida.

19.6 Estatísticas dos Programas BPF

Para avaliar a sobrecarga imposta por cada política, o Tetragon expõe as estatísticas dos programas BPF em uma visualização semelhante à do `top`:

```
docker exec -ti tetragon tetra debug progs
```

Nota: o subcomando `debug` não existe na CLI v1.7.1. Verifique com `tetra --help` se a sua versão o possui; caso não possua, use as métricas descritas abaixo.

Com o servidor de métricas habilitado, os mesmos dados ficam disponíveis nas métricas `tetragon_overhead_program_runs_total` e `tetragon_overhead_program_seconds_total`, rotuladas por política, sensor e ponto de conexão:

```
curl -s http://localhost:2112/metrics | grep tetragon_overhead
```

As estatísticas não são habilitadas por padrão, pois impõem sobrecarga adicional. Habilite-as apenas durante a medição:

```
sudo sysctl kernel.bpf_stats_enabled=1
sudo sysctl kernel.bpf_stats_enabled=0
```

19.7 Coleta de Dados para Análise

A ferramenta `bugtool` reúne, em um único arquivo, as informações do ambiente úteis para depuração, incluindo a configuração em uso, as políticas carregadas e o estado dos programas BPF:

```
docker exec tetragon tetra bugtool
```

Nota: as páginas de referência para esta seção são [Troubleshooting](#) e [FAQ de instalação](#).

20 Proteção da API gRPC com TLS

Seção pendente: pesquisar o procedimento em [gRPC TLS / mTLS](#) e documentar a configuração dos certificados no ambiente em contêiner.